Chapter 5 — Matrices and Equations: 80/20 Interview Notes

0. What you should actually master

Concept	Priority	Interview importance
$Ax = b$representation	**P0**	⭐⭐⭐⭐⭐
Gaussian elimination	**P0**	⭐⭐⭐⭐⭐
Row operations	**P0**	⭐⭐⭐⭐⭐
Back substitution	**P0**	⭐⭐⭐⭐⭐
Unique / zero / infinite solutions	**P0**	⭐⭐⭐⭐⭐
Singular vs invertible matrix	**P0**	⭐⭐⭐⭐⭐
LU decomposition	**P0**	⭐⭐⭐⭐⭐
Triangular matrices	**P0**	⭐⭐⭐⭐
Complexity $O(n^3)$	**P0**	⭐⭐⭐⭐⭐
Determinant through LU	**P0**	⭐⭐⭐⭐
Matrix inverse	**P1**	⭐⭐⭐⭐
Principal minors	**P1**	⭐⭐
Recursive determinant	**P2**	⭐
Hand calculation of large systems	**P2**	⭐

Important interview correction: In real numerical computing, you generally **do not explicitly compute** A^{-1}just to solve $Ax = b$. You normally use an appropriate factorization/solver such as LU, QR, or Cholesky. Explicit inversion can be less efficient and less numerically desirable.

---

1. Linear Equations

Definition

A **linear equation** is an equation where every unknown appears only to the first power and variables are not multiplied together.

Example:

$$2x + 3y = 10$$

is linear.

But:

$$x^2 + y = 10$$

is not linear.

And:

$$xy = 10$$

is not linear.

---

2. System of Linear Equations

A collection of linear equations sharing the same unknowns is a **system of linear equations**.

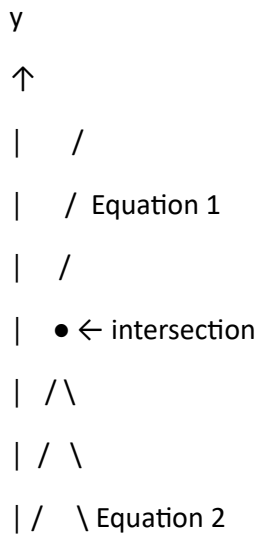
Example:

$$x + y = 5$$
$$2x + 3y = 12$$

We want values of x, ysatisfying **both equations simultaneously**.

Geometric intuition

For two variables, each equation represents a line.



└————————→ x

The intersection is the solution.

80/20 takeaway

A system of equations is fundamentally asking:

Can we find an $x$such that all constraints are satisfied simultaneously?

This perspective becomes extremely important in ML optimization, regression, numerical computing, and scientific computing.

---

3. Matrix Form: $\boldsymbol{Ax = b}$

This is probably the **single most important concept in this chapter**.

Suppose:

$$x_1 - 2x_2 = 0$$
$$20x_1 + 50x_2 = 900$$

We can write:

$$\begin{bmatrix} 1 & -2 \\ 20 & 50 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 900 \end{bmatrix}$$

or:

$$\boxed{Ax = b}$$

where:

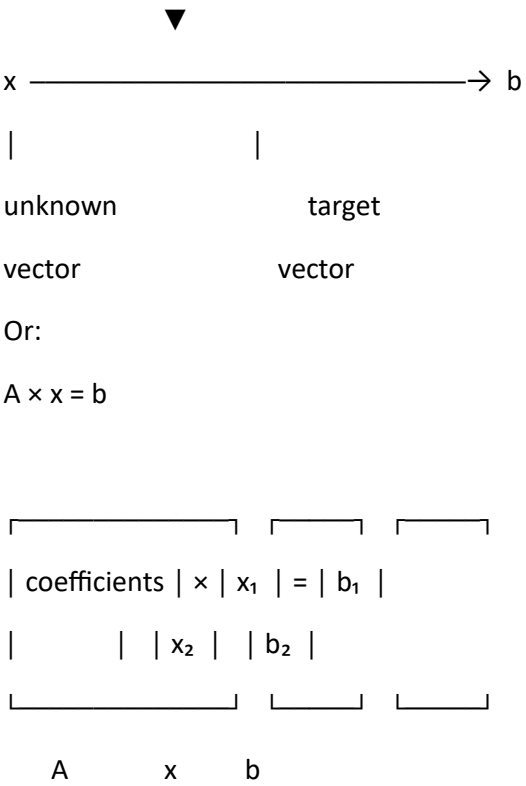
$$A = \begin{bmatrix} 1 & -2 \\ 20 & 50 \end{bmatrix}$$
$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$
$$b = \begin{bmatrix} 0 \\ 900 \end{bmatrix}$$

Diagram

Linear transformation

A

|



Interview answer

If asked:

What does $Ax = b$ mean?

Say:

$A$is the coefficient matrix, $x$contains the unknown variables, and $b$contains the right-hand-side values. Solving $Ax = b$means finding the vector $x$that the linear transformation represented by Amaps to b.

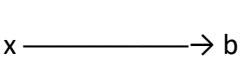
That's a strong answer.

---

4. Solving $\boldsymbol{Ax = b}$

There are three possible outcomes.

Case 1 — Unique solution



Exactly ONE x produces b.

Case 2 — No solution

b lies outside the image/column space of A

No x produces b.

Case 3 — Infinitely many solutions

Several different x values

produce the same b.

So:

0 solutions, 1 solution, or infinitely many solutions

for a linear system.

---

5. Gaussian Elimination ⭐⭐⭐⭐⭐

Definition

Gaussian elimination is an algorithm that transforms a system of linear equations into an upper-triangular form so that the unknowns can be solved using back substitution.

This is one of the most important algorithms in computational linear algebra.

---

6. Why Gaussian Elimination?

Suppose:

$$x + y + z = 6$$
$$2x + y + 5z = 2$$
$$-2x - 3y + 8z = 2$$

Trying to solve these simultaneously is messy.

Gaussian elimination transforms this:

Original

[1 1 1 | 6]

[2 1 5 | 2]

[-2 -3 8 | 2]

into:

After elimination

[1 1 1 | 6]

[0 -1 3 | -10]

[0 0 -? | ?]

The matrix becomes triangular.

Then solve **from bottom to top**.

---

7. Elementary Row Operations

Gaussian elimination relies on operations that preserve the solution set.

Operation 1 — Swap rows

$$R_1 \leftrightarrow R_2$$

Operation 2 — Multiply a row by nonzero scalar

$$R_1 \leftarrow cR_1,\ c \neq 0$$

Operation 3 — Add multiple of one row to another

$$R_2 \leftarrow R_2 + cR_1$$

These operations don't change which $x$solves the system.

---

8. Gaussian Elimination — Example

Consider:

$$x + y = 5$$
$$2x + 3y = 12$$

Augmented matrix:

$$\begin{bmatrix} 1 & 1 & 5 \\ 2 & 3 & 12 \end{bmatrix}$$

Eliminate $x$from row 2:

$$R_2 \leftarrow R_2 - 2R_1$$

giving:

$$\begin{bmatrix} 1 & 1 & 5 \\ 0 & 1 & 2 \end{bmatrix}$$

Therefore:

$$y = 2$$

Then:

$$x + y = 5$$

so:

$$x = 3$$

Solution

$$\boxed{x = 3,\ y = 2}$$

---

9. Gaussian Elimination Algorithm

The general idea:

Input A, b

|

▼

Choose pivot

|

▼

Eliminate entries below pivot

|

▼

Move to next column

|

▼

Repeat

|

▼

Upper triangular matrix

|

▼

Back substitution

|

▼

Solution x

---

10. Pivot

The current diagonal element used to eliminate values below it is called a **pivot**.

Example:

[2 4 5]

[6 7 8]

[3 1 9]

 ↑

pivot

Here 2is the first pivot.

---

11. The Zero Pivot Problem

Suppose:

$$\begin{bmatrix} 0 & 2 \\ 3 & 4 \end{bmatrix}$$

You cannot divide by the first pivot because:

$$a_{11} = 0$$

But the system might still be perfectly solvable.

We can swap rows:

[0 2] [3 4]

[3 4] → [0 2]

This is why practical Gaussian elimination uses **pivoting**.

---

12. Partial Pivoting ⭐⭐⭐⭐⭐

This is more important in practical numerical computing than the book's simplified version.

Instead of blindly using the diagonal element as the pivot, choose a good pivot from the remaining column.

For example:

[0.000001 2]

[5 3]

Using 0.000001as a pivot can create huge numerical errors.

Instead, swap:

[5 3]

[0.000001 2]

Why?

Because floating-point arithmetic has limited precision.

So practical LU decomposition usually uses:

$$\boxed{PA = LU}$$

where $P$is a permutation matrix representing row swaps.

Interview point

If asked:

Why is pivoting necessary?

Answer:

Pivoting avoids zero or very small pivots, which can cause division-by-zero or severe numerical instability. Partial pivoting swaps rows so that a larger-magnitude element is used as the pivot.

This is **very good interview knowledge**.

---

13. Back Substitution

After elimination we get:

$$Ux = y$$

where $U$is upper triangular.

Example:

$$\begin{aligned} 2x + 3y + z &= 10 \\ 4y + 2z &= 8 \\ 5z &= 10 \end{aligned}$$

Start at the bottom:

$$5z = 10$$

so:

$$z = 2$$

Then:

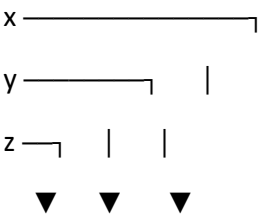
$$\begin{aligned} 4y + 2(2) &= 8 \\ y &= 1 \end{aligned}$$

Finally:

$$\begin{aligned} 2x + 3(1) + 2 &= 10 \\ x &= 2.5 \end{aligned}$$

Diagram

Upper triangular system



---

14. Complexity of Gaussian Elimination ⭐⭐⭐⭐⭐

The complexity is:

$$\boxed{O(n^3)}$$

Why?

For each pivot, we perform roughly quadratic work, and there are $n$pivots:

$$n \times n^2 = n^3$$

More precisely, the leading term is approximately:

$$\frac{2}{3}n^3$$

floating-point operations for standard dense LU/Gaussian elimination.

Interview question

Is $O(n^3)$good?

For dense linear algebra, yes—it is polynomial and highly optimized in practice.

Libraries can exploit:

- CPU vectorization
- cache locality
- multithreading

- BLAS
- LAPACK
- optimized hardware

This is why writing Python loops yourself can be dramatically slower than calling optimized numerical libraries.

---

15. Singular vs Invertible Matrix ⭐⭐⭐⭐⭐

This is critical.

A square matrix $A$is **invertible** if there exists:

$$A^{-1}$$

such that:

$$AA^{-1} = A^{-1}A = I$$

If no inverse exists, $A$is **singular**.

Invertible

$$\det(A) \neq 0$$

Equivalent ideas:

det(A) ≠ 0

↓

A invertible

↓

Ax=b has exactly one solution

for every b

↓

ker(A) = {0}

↓

columns are linearly independent

These are deeply connected.

---

16. Singular Matrix

Example:

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

Second row is:

$$2 \times \text{first row}$$

Therefore rows are dependent.

Its determinant:

$$\det(A) = 1(4) - 2(2) = 0$$

So:

$$\boxed{A \text{ is singular}}$$

---

17. The Three Solution Cases

For:

$$Ax = b$$

Unique

det(A) ≠ 0

A

x ——————→ b

one x

No solution

A is singular

+

b is incompatible with A

no x

Infinite solutions

A is singular

+

system is consistent

many x

---

18. Kernel Connection ⭐⭐⭐⭐⭐

The **kernel/null space** is:

$$\ker(A) = \{x : Ax = 0\}$$

Suppose:

$$Ax = b$$

and $x_0$is one solution.

Then **every solution** is:

$$\boxed{x = x_0 + y,\ y \in \ker(A)}$$

or:

$$\boxed{x_0 + \ker(A)}$$

This is one of the most beautiful results in the chapter.

Why?

If:

$$Ax_0 = b$$

and:

$$Ay = 0$$

then:

$$\begin{aligned} &A(x_0 + y) \\ &= Ax_0 + Ay \\ &= b + 0 \\ &= b \end{aligned}$$

Therefore $x_0 + y$is also a solution.

---

19. Example of Infinite Solutions

Consider:

$$\begin{aligned} x + y &= 5 \\ 2x + 2y &= 10 \end{aligned}$$

The second equation contains no new information.

So:

$$x + y = 5$$

Choose:

$$y = t$$

Then:

$$x = 5 - t$$

Therefore:

$$\boxed{x = \begin{bmatrix} 5 - t \\ t \end{bmatrix}}$$

for any t.

There are infinitely many solutions.

---

20. LU Decomposition ⭐ ⭐ ⭐ ⭐ ⭐

This is the second major concept.

Definition

LU decomposition factors a matrix into:

$$\boxed{A = LU}$$

where:

- L= lower triangular matrix
- U= upper triangular matrix

Example:

$$A = \begin{bmatrix} 4 & 3 \\ 6 & 3 \end{bmatrix}$$

can be decomposed into:

$$L = \begin{bmatrix} 1 & 0 \\ 1.5 & 1 \end{bmatrix}$$

and:

$$U = \begin{bmatrix} 4 & 3 \\ 0 & -1.5 \end{bmatrix}$$

Check:

$$LU = \begin{bmatrix} 4 & 3 \\ 6 & 3 \end{bmatrix} = A$$

---

21. Why LU?

This is the important part.

Suppose you need to solve:

$$Ax = b_1$$

and later:

$$Ax = b_2$$

and:

$$Ax = b_3$$

If Adoesn't change, repeatedly performing Gaussian elimination is wasteful.

Factor once:

$$A = LU$$

Then:

$$LUx = b$$

Let:

$$Ly = b$$

First solve:

$$Ly = b$$

Then:

$$Ux = y$$

So:

A x = b

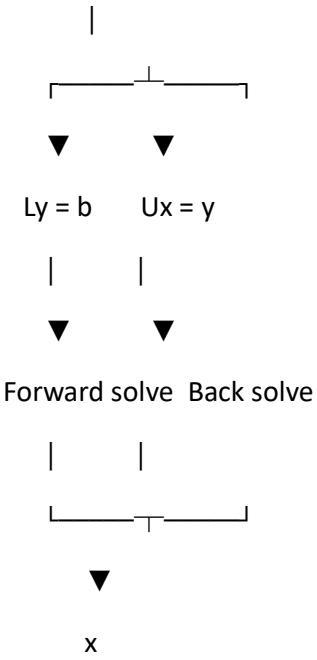
|

A = LU

|

▼

LUx = b



---

22. The Real Power of LU

Suppose:

$$A$$

is a 1000×1000matrix.

You have 10,000 different $b$vectors.

Instead of factorizing A10,000 times:

A

|

▼

LU decomposition

|

├— solve b_1

├— solve b_2

├— solve b_3

├— ...

└── solve b_{10000}

The expensive factorization is performed **once**.

This is a major practical reason LU matters.

---

23. LU Complexity

Factorization:

$$\boxed{O(n^3)}$$

Forward substitution:

$$O(n^2)$$

Back substitution:

$$O(n^2)$$

Therefore:

Factorization $O(n^3)$

+

Each solve $O(n^2)$

This becomes extremely useful when the same $A$is used with many b's.

---

24. Forward Substitution

For:

$$Ly = b$$

where $L$is lower triangular:

[● 0 0] [y1] [b1]

[● ● 0] [y2] = [b2]

[● ● ●] [y3] [b3]

Solve from:

$$y_1$$

downward.

---

25. Back Substitution

For:

$$Ux = y$$

where $U$is upper triangular:

[● ● ●] [x1] [y1]

[0 ● ●] [x2] = [y2]

[0 0 ●] [x3] [y3]

Solve from:

$$x_n$$

upward.

---

26. LU and Gaussian Elimination Relationship

This is an important interview question.

Gaussian elimination

Transforms:

$$A \rightarrow U$$

LU

Stores the elimination information in L:

$$\boxed{A = LU}$$

So LU is essentially a **structured representation of Gaussian elimination**.

Think:

Gaussian elimination:

A ——————————→ U

elimination

LU decomposition:

A ——————————→ L + U

|

└── elimination factors stored in L

---

27. Implementation — LU from Scratch

For learning purposes, a simple implementation is extremely valuable.

```
import numpy as np

def lu_decomposition(A):
    A = A.astype(float)
    n = A.shape[0]

    L = np.eye(n)
    U = A.copy()

    for k in range(n - 1):

        # Pivot
        if U[k, k] == 0:
            raise ValueError("Zero pivot encountered")
```

```
        for i in range(k + 1, n):

            multiplier = U[i, k] / U[k, k]

            L[i, k] = multiplier

            U[i, k:] -= multiplier * U[k, k:]

    return L, U
```

Example:

```
A = np.array([
    [4, 3],
    [6, 3]
])

L, U = lu_decomposition(A)

print(L)
print(U)

print(np.allclose(L @ U, A))
```

Expected:

True

---

28. But This Implementation Has a Problem

The code above doesn't perform pivoting.

A production-quality dense LU implementation generally needs pivoting.

Conceptually:

if abs(U[k, k]) is too small:

find largest suitable pivot

swap rows

The factorization becomes:

$$\boxed{PA = LU}$$

rather than simply:

$$A = LU$$

This distinction is **very interview-worthy**.

---

29. Solving $Ax = b$ Using LU

Suppose:

$$A = LU$$

We have:

$$Ax = b$$

Therefore:

$$LUx = b$$

Define:

$$y = Ux$$

Actually, more conveniently:

$$Ly = b$$

then:

$$Ux = y$$

Python conceptually:

L, U = lu_decomposition(A)

y = forward_substitution(L, b)

x = back_substitution(U, y)

This is how the factorization becomes a solver.

---

30. Determinant ⭐ ⭐ ⭐ ⭐

Definition

The determinant is a scalar associated with a square matrix.

It tells us several things, including whether the matrix is invertible.

$$\boxed{\det(A) \neq 0 \iff A \text{ is invertible}}$$

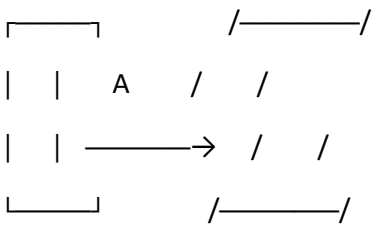
---

31. Geometric Meaning of Determinant

The determinant measures **signed volume scaling** of a linear transformation.

In 2D:

Original square Transformed parallelogram



If:

$$|\det(A)| = 3$$

areas are scaled by a factor of 3.

In 3D, volume is scaled.

---

32. Determinant of Triangular Matrix

If:

$$U = \begin{bmatrix} u_{11} & * & * \\ 0 & u_{22} & * \\ 0 & 0 & u_{33} \end{bmatrix}$$

then:

$$\boxed{\det(U) = u_{11}u_{22}u_{33}}$$

In general:

$$\boxed{\det(U) = \prod_i u_{ii}}$$

This is extremely useful with LU.

---

33. Determinant Through LU

If:

$$A = LU$$

then:

$$\det(A) = \det(L)\det(U)$$

For standard LU where Lhas 1s on its diagonal:

$$\det(L) = 1$$

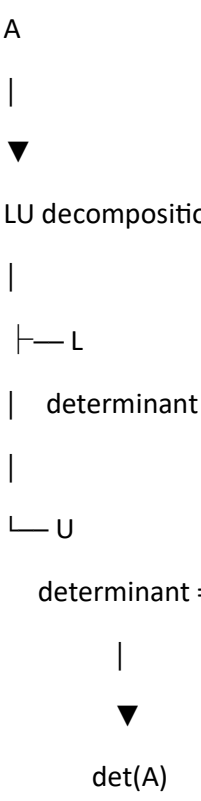
Therefore:

$$\boxed{\det(A) = \det(U)}$$

and:

$$\boxed{\det(A) = \prod_i U_{ii}}$$

Diagram



With pivoting $PA = LU$, remember the permutation changes the determinant sign:

$$\det(A) = \det(P)^{-1}\det(L)\det(U)$$

and since $\det(P) = \pm 1$, row swaps affect the sign.

---

34. Recursive Determinant vs LU

The textbook demonstrates why naive recursive determinant calculation is terrible.

For example:

$$\det(A) = \sum_{j} (-1)^{j+1} a_{1j} \det\left(A_{1j}\right)$$

This recursively creates many subproblems.

Approximate complexity:

$$\boxed{O(n!)}$$

That becomes unusable very quickly.

Comparison

Method	**Approx. complexity**
Recursive determinant	$O(n!)$
LU-based determinant	$O(n^3)$

This is a great example of why algorithm selection matters.

---

35. Matrix Inverse

Definition

The inverse A^{-1}satisfies:

$$\boxed{AA^{-1} = A^{-1}A = I}$$

provided $A$is invertible.

Then:

$$Ax = b$$

can theoretically be solved as:

$$\boxed{x = A^{-1}b}$$

---

36. How Do We Compute an Inverse?

A naive conceptual approach is to solve:

$$\begin{aligned} Ax_1 &= e_1 \\ Ax_2 &= e_2 \\ &\dots \\ Ax_n &= e_n \end{aligned}$$

where:

$$e_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

etc.

Then:

$$A^{-1} = \begin{bmatrix} | & | & & | \\ x_1 & x_2 & \cdots & x_n \\ | & | & & | \end{bmatrix}$$

Diagram

A $x_1 = e_1 \longrightarrow x_1$

A $x_2 = e_2 \longrightarrow x_2$

A $x_3 = e_3 \longrightarrow x_3$

...

A $x_n = e_n \longrightarrow x_n$

↓

$A^{-1} = [x_1\ x_2\ \ldots\ x_n]$

---

37. Important Production Rule

Don't say:

"To solve $Ax = b$, always calculate A^{-1}."

That's mathematically valid but computationally poor.

Prefer:

Need x from Ax=b

|

▼

Use a linear solver/factorization

|

├— LU

├── QR

├── Cholesky

└── iterative methods

rather than explicitly calculating:

$$A^{-1}$$

unless the inverse itself is actually required.

---

38. Why Avoid Explicit Inverse?

Suppose:

$$Ax = b$$

Computing:

$$A^{-1}$$

and then:

$$x = A^{-1}b$$

does extra work.

A solver can directly obtain x.

Conceptually:

Explicit inverse:

A → A^{-1} → multiply by b

↑

extra work

Direct solve:

A → factorize → solve

This is a practical numerical linear algebra principle.

---

39. NumPy Implementation

For learning:

```
A = np.array([
    [2, 1],
    [1, 3]
], dtype=float)


b = np.array([5, 6], dtype=float)


x = np.linalg.solve(A, b)


print(x)
```

Prefer:

np.linalg.solve(A, b)

over:

np.linalg.inv(A) @ b

when your goal is simply to solve $Ax = b$.

---

40. Why NumPy Is So Fast

Your textbook's experiment demonstrates something very important.

Pure Python implementation:

many Python-level loops

↓

slow

Optimized numerical library:

NumPy

↓

BLAS/LAPACK

↓

optimized native code

↓

CPU vectorization/cache/multithreading

↓

much faster

This is a major engineering lesson:

Algorithmic complexity matters, but implementation and hardware utilization matter too.

---

41. Complexity Cheat Sheet

Memorize this table.

Operation	**Dense matrix complexity**
Matrix-vector multiplication	$O(n^2)$
Matrix-matrix multiplication	$O(n^3)$
Gaussian elimination	$O(n^3)$
LU factorization	$O(n^3)$
Forward substitution	$O(n^2)$
Back substitution	$O(n^2)$
Solve after LU	$O(n^2)$per RHS
Explicit inverse	$O(n^3)$
Recursive determinant	roughly $O(n!)$
Determinant via LU	$O(n^3)$

---

42. Principal Minors — Understand, Don't Over-Memorize

A **principal minor** is the determinant of the upper-left $k \times k$submatrix.

For:

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

we have:

$$A_1 = [a]$$
$$A_2 = \begin{bmatrix} a & b \\ d & e \end{bmatrix}$$
$$A_3 = A$$

and:

$$M_k = \det(A_k)$$

The chapter uses these to characterize when **Gaussian elimination without row pivoting** can proceed without zero pivots.

Interview priority

Low-to-medium.

Know:

Principal minors help characterize whether the particular no-pivot Gaussian elimination process encounters zero pivots.

Don't spend hours memorizing the theorem unless your interview specifically targets numerical linear algebra.

---

43. Important Correction About the Book's Theorem

Be careful with this statement:

"If $\det A \neq 0$, then all principal minors are nonzero."

That statement is **not true for arbitrary matrices**.

For example:

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

has:

$$\det(A) = -1 \neq 0$$

but:

$$M_1 = 0$$

So $A$is invertible, yet the first pivot is zero.

This is exactly why **row pivoting** matters.

The more useful practical statement is:

$$\boxed{\det(A) \neq 0 \Rightarrow A \text{ is invertible}}$$

but it does **not** guarantee that naive Gaussian elimination without row swaps succeeds.

That distinction is worth knowing for interviews.

---

44. LU Without Pivoting vs LU With Pivoting

No pivoting

$$\boxed{A = LU}$$

Requires suitable nonzero pivots.

With row pivoting

$$\boxed{PA = LU}$$

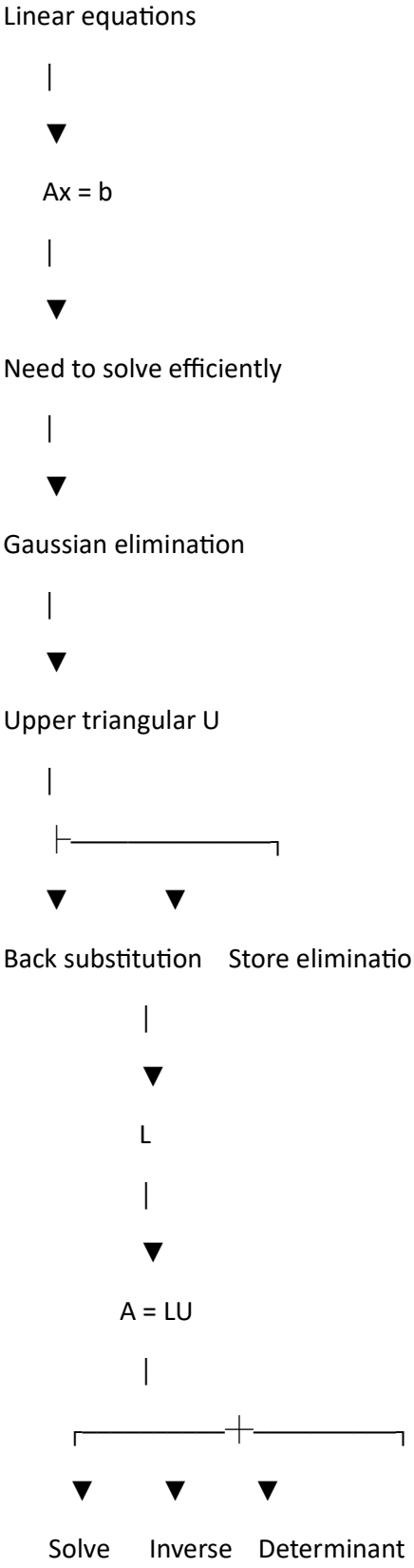
where Precords row permutations.

In numerical computing, the second form is generally the safer default.

---

45. Production-Oriented Mental Model

You should think of the chapter like this:



That's the **entire chapter in one diagram**.

---

46. AI/ML Connection ⭐⭐⭐⭐⭐

Since your goal includes AI/ML engineering, this chapter is absolutely relevant.

Linear regression

A basic linear regression model can be written:

$$Xw \approx y$$

The classical least-squares solution leads to:

$$X^T Xw = X^T y$$

So we get a linear system:

$$\boxed{Aw = b}$$

where:

$$A = X^T X$$

and:

$$b = X^T y$$

Therefore:

Machine Learning

|

▼

Linear Regression

|

▼

Solve linear system

|

▼

Linear Algebra

|

▼

LU / QR / Cholesky / iterative solvers

This is one reason linear algebra is fundamental to ML.

---

47. Important ML Warning

Although you may encounter:

$$w = (X^T X)^{-1} X^T y$$

in textbooks, in numerical implementations you generally don't want to explicitly compute:

$$(X^T X)^{-1}$$

and you often don't want to form $X^T X$ unnecessarily either, because it can worsen conditioning.

More stable approaches include:

QR decomposition

or other appropriate numerical solvers.

This is a **very good AI/ML interview distinction**:

Mathematical formula ≠ best numerical implementation.

---

48. Real-World Example: Recommendation System

Suppose you want to estimate user preferences:

User 1 → likes products A, B, C

User 2 → likes A, C

User 3 → likes B, D

...

You can build matrices representing:

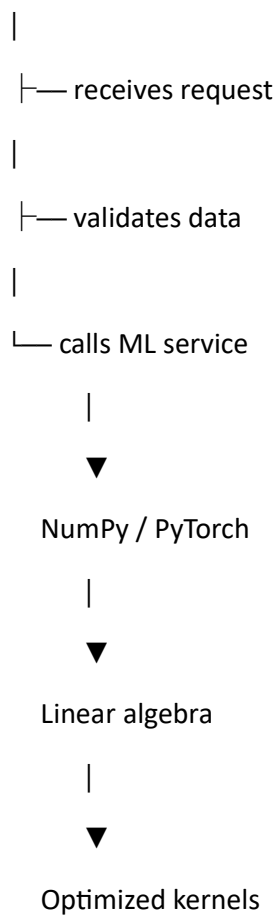
$$Xw \approx y$$

and solving for $w$becomes a linear algebra problem.

The backend may ultimately call a numerical library rather than implementing Gaussian elimination itself.

This is where your Java backend and AI/ML paths connect:

Java Backend



---

49. Interview Example: Cash Dispenser

The textbook's example is useful because it demonstrates modeling.

Given:

- 20-dollar bills = x_1
- 50-dollar bills = x_2
- total = $900

- twice as many $20 bills

We get:

$$x_1 - 2x_2 = 0$$
$$20x_1 + 50x_2 = 900$$

Matrix:

$$A = \begin{bmatrix} 1 & -2 \\ 20 & 50 \end{bmatrix}$$

Then:

$$Ax = b$$

Solution:

$$x_2 = 10$$
$$x_1 = 20$$

Interview lesson

The important thing isn't the arithmetic.

It's:

Translate a real-world constraint system into matrix form.

---

50. Interview Example: Database/System Capacity

Imagine a backend has two server types:

- Type A costs $100/month
- Type B costs $200/month

You need:

- 10 servers
- $1,500 budget

Then:

$$x + y = 10$$
$$100x + 200y = 1500$$

Again:

$$Ax = b$$

The same mathematical machinery applies.

This demonstrates why linear systems are more than "math homework": they're a way to represent simultaneous constraints.

---

51. Interview Example: Network/Traffic

Suppose traffic enters a network and must satisfy conservation constraints.

You might get:

$$Ax = b$$

where:

- x= unknown flows
- A= network constraints
- b= observed demands

Solving the system gives the unknown flows.

This idea appears in:

- network optimization
- electrical circuits
- fluid systems
- operations research
- ML
- scientific computing

---

52. Interview Example: Multiple RHS

Suppose:

$$AX = B$$

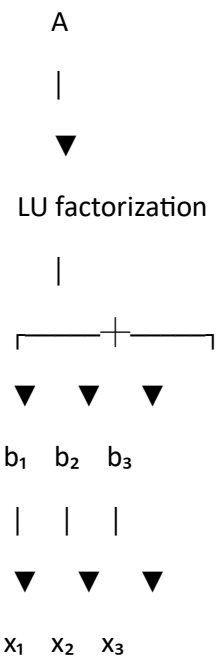
where Bcontains many columns:

$$B = \begin{bmatrix} | & | & | \\ b_1 & b_2 & b_3 \\ | & | & | \end{bmatrix}$$

Then:

$$AX = B$$

can be solved efficiently after one LU factorization.



This is extremely useful in numerical applications.

---

53. Implementation You Should Know for Interviews

You do **not** need to memorize the textbook's exact elimination_matrix() implementation.

Instead, understand this pseudocode:

LU(A):

L = identity

U = copy(A)

```
for k = 0 ... n-2:

    choose pivot

    for i = k+1 ... n-1:

        multiplier = U[i][k] / U[k][k]

        L[i][k] = multiplier

        eliminate:
            U[i][j] -= multiplier * U[k][j]

return L, U
```

The key line is:

$$\boxed{U_{ij} \leftarrow U_{ij} - \frac{U_{ik}}{U_{kk}} U_{kj}}$$

That's the mathematical heart of Gaussian elimination.

---

54. Don't Memorize This

You don't need to memorize:

elim_mtx[step:, step-1] = -a[step:]

Instead understand:

elimination matrix

↓

performs row operations

↓

creates zeros below pivot

↓

produces U

And:

inverse elimination factors

↓

store multipliers

↓

produce L

That's much more valuable in interviews.

---

55. Most Important Interview Questions

Q1. What is $Ax = b$?

Answer:

It represents a system of linear equations in matrix form, where $A$is the coefficient matrix, $x$is the vector of unknowns, and $b$is the right-hand side. Solving the system means finding the vector $x$mapped to $b$by A.

Q2. What is Gaussian elimination?

An algorithm that uses elementary row operations to transform a linear system into upper-triangular form, after which back substitution finds the solution.

Q3. Why is Gaussian elimination $O(n^3)$?

There are roughly $n$elimination stages, and each stage performs quadratic work on the remaining submatrix, giving $O(n^3)$.

Q4. What is LU decomposition?

It factors a matrix into a lower triangular matrix and an upper triangular matrix: $A = LU$, or $PA = LU$ when pivoting is used.

Q5. Why is LU useful?

If the same matrix A must be used to solve many systems with different right-hand sides, we can factor A once and reuse the factorization. Each subsequent solve costs only $O(n^2)$.

Q6. Why do we need pivoting?

To avoid zero or very small pivots, which can cause division by zero or numerical instability. Partial pivoting swaps rows to select a better pivot.

Q7. When does $Ax = b$ have a unique solution?

For square A:

$$\boxed{\det(A) \neq 0}$$

equivalently:

$$A \text{ is invertible}$$

and:

$$\ker(A) = \{0\}$$

Q8. What happens when A is singular?

Depending on b, the system can have either no solution or infinitely many solutions.

Q9. Why is the determinant of triangular U easy?

Because the determinant of a triangular matrix equals the product of its diagonal elements.

Q10. How does LU help calculate determinant?

$$A = LU$$

Therefore:

$$\det(A) = \det(L)\det(U)$$

If Lhas unit diagonal:

$$\det(L) = 1$$

so:

$$\boxed{\det(A) = \prod_i U_{ii}}$$

with the appropriate sign adjustment when row permutations are used.

Q11. Should you calculate $A^{-1}$to solve $Ax = b$?

Best answer:

Usually no. Although $x = A^{-1}b$is mathematically correct, numerical libraries generally use a factorization or direct solver because explicitly forming the inverse adds unnecessary computation and can be less numerically stable.

Q12. What is the relationship between Gaussian elimination and LU?

Gaussian elimination transforms Ainto U. LU decomposition additionally records the elimination multipliers in L, giving $A = LU$, or $PA = LU$with pivoting.

---

56. The 10 Things You Must Remember

If you remember only these:

1.

$$\boxed{Ax = b}$$

represents a system of linear equations.

2.

Gaussian elimination eliminates variables using row operations.

3.

Gaussian elimination produces:

$$\boxed{U}$$

an upper-triangular matrix.

4.

Then use back substitution.

5.

Dense Gaussian elimination/LU:

$$\boxed{O(n^3)}$$

6.

LU:

$$\boxed{A = LU}$$

or, with pivoting:

$$\boxed{PA = LU}$$

7.

Forward substitution solves:

$$Ly = b$$

8.

Back substitution solves:

$$Ux = y$$

9.

$$\boxed{\det(A) \neq 0 \iff A \text{ is invertible}}$$

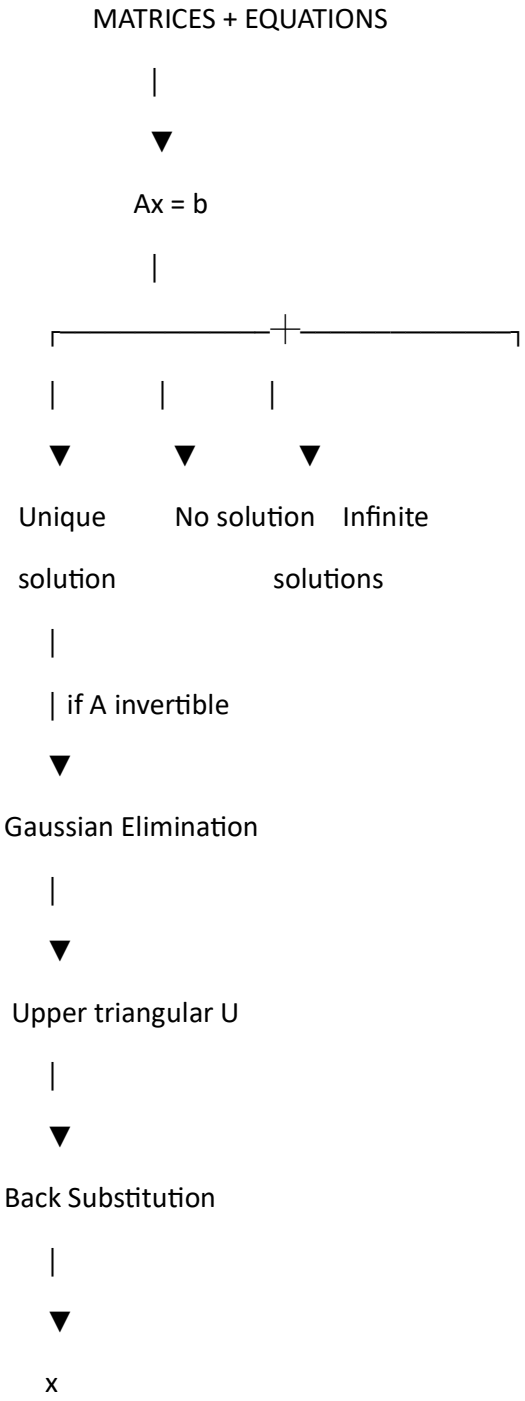
for square matrices.

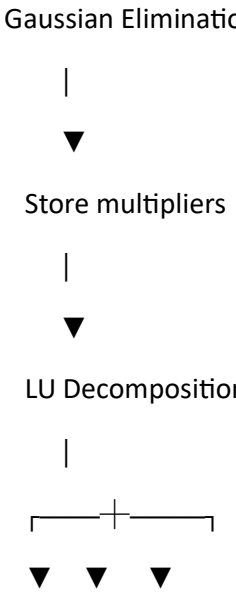
10.

For solving $Ax = b$, prefer a solver/factorization over explicitly calculating A^{-1}.

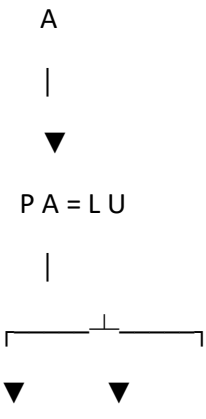
---

57. One-Page Mental Map

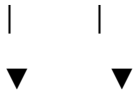




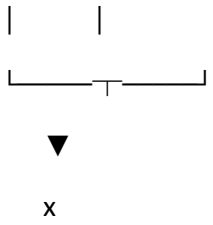
LU:



lower triangular upper triangular



forward solve back solve



Complexity:

Gaussian elimination → O(n³)

LU factorization → O(n³)

Forward substitution → O(n²)

Back substitution → O(n²)

Each solve after LU → O(n²)

---

58. Your 80/20 Study Order

For your interview preparation, I recommend learning this chapter in exactly this order:

P0 — Master

1. **What** $Ax = b$**means**
2. **Matrix representation of equations**
3. **Elementary row operations**
4. **Gaussian elimination**
5. **Pivoting**
6. **Back substitution**
7. **Unique/no/infinite solutions**
8. **Singular vs invertible**
9. **LU decomposition**
10. **Forward + backward substitution**
11. $O(n^3)$**complexity**

12. **Why direct solving is preferable to explicit inverse**

P1 — Understand

13. Determinant
14. Determinant through LU
15. Kernel and solution space
16. Multiple right-hand sides
17. Numerical stability
18. Partial pivoting / $PA = LU$

P2 — Don't spend much time

19. Recursive determinant implementation
20. Principal-minor formulas
21. Memorizing individual elimination matrices
22. Reproducing every proof from the chapter by memory

---

59. Final Interview-Level Understanding

The deepest takeaway from this chapter isn't:

"I know Gaussian elimination."

It is this chain:

$$\boxed{\text{Real-world constraints} \rightarrow Ax = b \rightarrow \text{Gaussian elimination} \rightarrow LU \rightarrow \text{efficient solving}}$$

And then:

$$\boxed{LU \rightarrow \begin{cases} \text{solve systems} \\ \text{compute determinant} \\ \text{compute inverse} \\ \text{reuse factorization} \end{cases}}$$

And for AI/ML:

$$\boxed{\text{ML models} \rightarrow \text{linear algebra} \rightarrow \text{linear systems} \rightarrow \text{numerical solvers}}$$

That is the **80/20 knowledge you should carry into your Java Backend + AI/ML engineering path**.

Mastery gate

Before moving to the next chapter, you should be able to explain **without notes**:

1. Why $Ax = b$represents simultaneous equations.
2. How Gaussian elimination creates zeros below the pivot.
3. Why back substitution works.
4. Why the complexity is $O(n^3)$.
5. What singular and invertible mean.
6. Why $PA = LU$is more practical than plain $A = LU$.
7. Why LU is valuable when solving many systems with the same A.
8. How determinant relates to invertibility.
9. Why explicitly computing $A^{-1}$is usually not the preferred way to solve $Ax = b$.
10. How all of this connects to linear regression and ML.

If you can **derive and explain those 10 points rather than memorize them**, you've extracted most of the interview value from this chapter.